AutoScholarLoop

A Practical AI Paper Writing Agent and Multi-Agent Research Automation System

Project Overview and Demo Whitepaper

AI paper writing | research automation | auditable artifacts | LaTeX output

Purpose of this document. This demo whitepaper introduces AutoScholarLoop as a GitHub project and runnable research automation system. It is not a scientific submission, does not report benchmark results, and does not claim that the system can replace human research judgment. Its goal is to explain what the project does, how the web/runtime architecture is organized, and how an initial research idea can become auditable drafts, checkpoints, and release artifacts.

Executive Summary

AutoScholarLoop is an open-source framework for AI-assisted research automation. It organizes the paper-writing process into a staged loop rather than a single chatbot interaction. A user provides a research seed, optional papers, notes, and configuration; the system then builds field context, proposes and critiques ideas, plans execution, drafts a manuscript, and runs quality checks. The output is not only a final draft, but also a local workspace containing intermediate files, checkpoints, review memos, LaTeX sources, and optional PDF artifacts.

What it is	A local, auditable AI paper-writing and research automation system.
Who it helps	Researchers, students, and developers who want a structured workflow for literature review, idea generation, execution planning, and manuscript drafting.
What it produces	Field summaries, idea critiques, execution plans, manuscript drafts, claim-evidence tables, quality reports, Markdown files, LaTeX files, and optional PDFs.
Why it matters	It makes AI-assisted research more inspectable by saving each stage as readable artifacts instead of hiding the process in one long chat.

1 Motivation and Project Positioning

Many researchers already use large language models for reading papers, brainstorming directions, drafting text, and polishing manuscripts. The problem is that these activities often happen in disconnected chats and scattered local files. It becomes difficult to track why an idea was chosen, which evidence supports each claim, what execution plan was proposed, and whether the final draft has passed basic quality checks.

AutoScholarLoop addresses this problem by turning AI-assisted paper writing into a staged and inspectable workflow. It should be understood as a **research automation scaffold**: it helps organize work, generate drafts, and record artifacts, while the user remains responsible for scientific validity, citation correctness, experimental verification, authorship, and venue compliance.

Positioning	Meaning
AI paper writing agent	Helps convert research materials into paper plans, claim-evidence tables, manuscript drafts, Markdown output, LaTeX output, and optional PDF output.
Research automation framework	Turns scattered research activities into a reproducible loop with stage-level checkpoints and auditable artifacts.
Multi-agent workflow	Separates work into field mapping, professor-style decision making, PhD-style execution planning, writing, and quality control.

2 Web and Runtime Architecture

Figure 1 is the recommended overview figure for this demo document. It explains how the web console, backend API, research loop engine, external providers, and local workspace fit together.

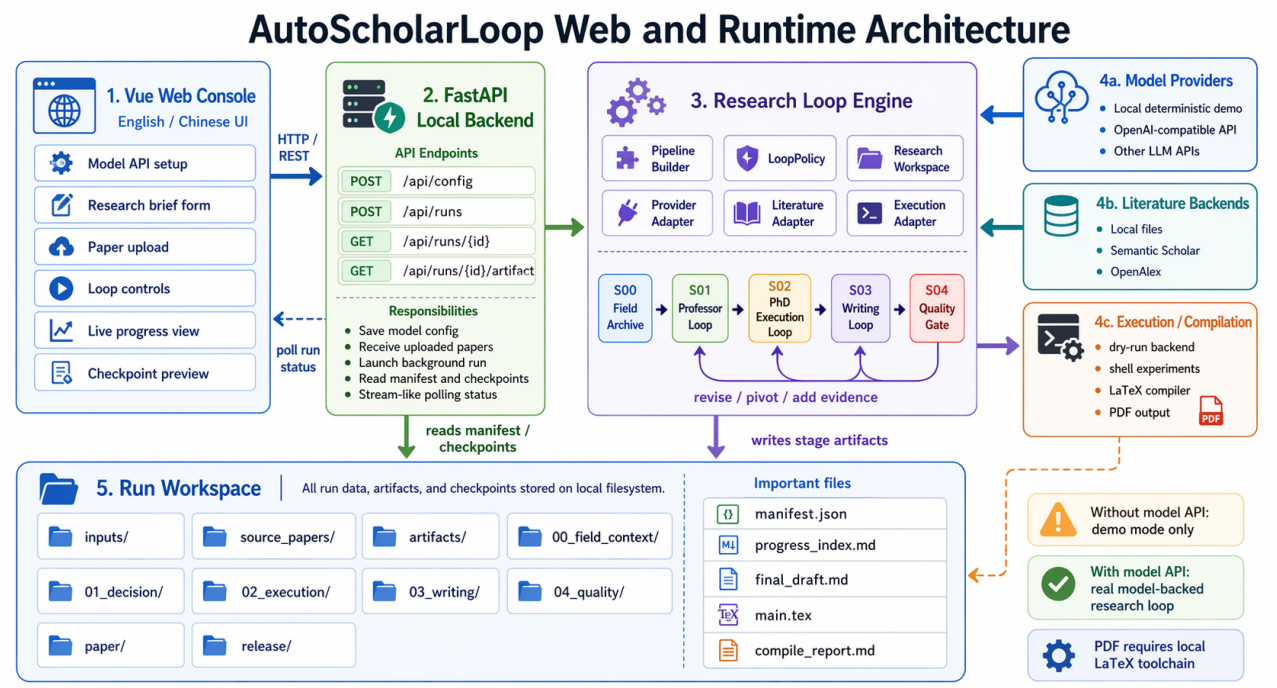

Figure 1. AutoScholarLoop web and runtime architecture. The system connects a Vue web console, a FastAPI local backend, a staged research loop engine, model/literature/execution adapters, and a local run workspace for auditable outputs.

The diagram can be read as one integrated runtime path. The user starts from the web console, where they configure model access, fill in a research brief, upload papers, control the loop, and inspect progress. The web console calls a local FastAPI backend through REST endpoints. The backend stores run configuration, receives uploaded files, launches runs, reads manifests and checkpoints, and returns progress or artifacts to the user interface.

The research loop engine is the core coordinator. It builds the pipeline, applies loop policies, connects model providers, accesses literature backends, invokes execution or compilation adapters, and writes outputs to the run workspace. The system can run in a deterministic demo mode without a real model API, or in a model-backed mode when the user configures an OpenAI-compatible or other supported model provider. PDF generation depends on the local LaTeX toolchain.

2.1 Main Components

Component	Role
Vue Web Console	Provides model API setup, research brief input, paper upload, loop controls, live progress view, and checkpoint preview.
FastAPI Local Backend	Stores model and run configuration, starts runs, reads manifests and checkpoints, and serves run status and artifacts.
Research Loop Engine	Coordinates the S00-S04 staged workflow and controls revision, pivot, evidence addition, or release.
Model Providers	Connect the loop to local deterministic demo mode or real model-backed inference.
Literature Backends	Support local source papers and external literature services.
Execution/Compilation Backends	Support dry-run execution, shell experiments, LaTeX compilation, and optional PDF output.
Run Workspace	Stores inputs, source papers, intermediate artifacts, logs, stage outputs, paper files, and release packages.

3 Research Loop and User Workflow

The staged loop is the central idea of AutoScholarLoop. Instead of asking one model to directly write a complete paper, the project decomposes the process into five inspectable stages. Each stage has its own responsibility and leaves behind files that can be reviewed, edited, or reused.

Stage	Name	Responsibility
S00	Field Archive	Builds field context, paper cards, method maps, dataset/baseline notes, and evidence banks.
S01	Professor Loop	Generates ideas, critiques them, checks novelty signals, ranks directions, and selects a candidate direction.
S02	PhD Execution Loop	Plans implementation, baselines, experiments, failure analysis, and execution review.
S03	Writing Loop	Builds a manuscript plan, claim-evidence table, draft, figure plan, and revision notes.
S04	Quality Gate	Audits novelty, citation quality, reproducibility, unsupported claims, and release readiness.

A typical user workflow is therefore simple at the surface but structured underneath:

1. Provide a research seed, target format, uploaded papers, notes, and optional code.
2. Create a local run workspace and save the run configuration.
3. Build field context and reusable evidence artifacts.
4. Generate, critique, and select a candidate research direction.
5. Plan execution and organize evidence for writing.
6. Draft the manuscript and produce Markdown/LaTeX artifacts.

7. Run quality checks and decide whether to release, revise, pivot, or add evidence.

This design makes the project useful not only for drafting text, but also for tracking the reasoning trail behind a draft. Users can inspect where claims came from, how ideas were selected, and what quality checks were applied before release.

4 Workspace, Artifacts, and Repository Layout

Every run writes data to a local workspace. This is a key difference between a normal chat-based writing assistant and AutoScholarLoop: the output is a directory of artifacts that can be reviewed, versioned, archived, or attached to a pull request.

```
run/
  inputs/
  source_papers/
  artifacts/
  00_field_context/
  01_decision/
  02_execution/
  03_writing/
  04_quality/
  paper/
  release/
  manifest.json
  progress_index.md
```

Typical important files include:

File	Meaning
`manifest.json`	Machine-readable record of run configuration, stage status, and artifact paths.
`progress_index.md`	Human-readable progress summary across stages.
`final_draft.md`	Main Markdown draft produced by the writing loop.
`main.tex`	LaTeX manuscript source generated from the draft and selected paper format.
`compile_report.md`	Notes on LaTeX compilation and PDF generation when a local toolchain is available.
`release/`	Final package for review or sharing.

At the repository level, the project can be introduced with the following simplified layout:

```
AutoScholarLoop/
  configs/ default configuration examples
  docs/ documentation and roadmap materials
  examples/ example inputs and runnable demos
  img/ project figures and screenshots
  src/ Python package and research loop implementation
  templates/ paper and artifact templates
  tests/ smoke tests and regression checks
  web/ Vue web console
```

5 Usage Modes and Extension Points

AutoScholarLoop supports two practical usage modes. The first is a demo mode, where users can inspect the workflow without configuring a real model API. This is useful for onboarding, documentation,

and repository demonstration. The second is a model-backed mode, where users configure a model provider and run a real research loop.

Mode	Description
Demo mode	Runs a deterministic local demonstration. It is useful for showing the pipeline structure, workspace layout, and artifact flow, but it is not a real model-backed research run.
Model-backed mode	Uses configured model providers to generate field summaries, ideas, execution plans, drafts, and quality reports. Human review is still required.

The project is designed around adapters, which makes it extensible. Model providers can be expanded to support additional LLM APIs. Literature adapters can add new academic search sources. Execution adapters can support different experiment runners or sandbox policies. Paper format templates can be extended to support more venues and publishers.

```
# Example command style. Adjust options according to the repository README.
python -m open_research_agent.cli run --config configs/demo.yaml

# Optional web usage pattern.
cd web
npm install
npm run dev
```

For a GitHub demo paper, the most important message is that AutoScholarLoop is not only a set of prompts. It is a runnable system that combines a user interface, a backend service, a staged loop engine, provider adapters, and filesystem artifacts.

6 Responsible Use and Recommended Repository Placement

AutoScholarLoop should be presented as an AI-assisted research and paper-writing workflow, not as an automatic paper-submission machine. The system can help with literature organization, idea generation, execution planning, manuscript drafting, and quality checks, but it cannot independently guarantee novelty, correctness, ethical compliance, authorship validity, citation accuracy, or acceptance by a venue.

Recommended responsible-use statements for the repository are:

- Users must verify all claims, citations, equations, experiments, and generated text.
- Generated drafts should be treated as review candidates, not final scientific results.
- The system should not be used to fabricate experiments, citations, authorship, or peer-review claims.
- Human researchers remain responsible for scientific judgment and submission decisions.

This document can be placed in the repository as a lightweight project whitepaper. A practical pull request could add:

```
docs/autoscholarloop_project_overview.tex
docs/autoscholarloop_project_overview.pdf
img/autoscholarloop_web_runtime_architecture.png
```

The value of the document is discoverability and onboarding. It gives new users a compact explanation of what AutoScholarLoop does, how the runtime is organized, why the staged loop matters, and what artifacts a run produces.

7 Conclusion

AutoScholarLoop is best understood as an auditable AI paper-writing and research automation system. Its main contribution as a project is not a single model or benchmark result, but an organized workflow that connects research context building, idea selection, execution planning, manuscript drafting, quality control, and artifact packaging. For GitHub users searching for AI paper writing agents, automatic manuscript drafting tools, or multi-agent research automation systems, this project overview can serve as the first readable entry point.